VIT · SY · ENGINEERING DESIGN & INNOVATION · MID-SEMESTER REVIEW

Autonomous Procurement Assistant

A multi-agent AI system that helps small Indian businesses buy things: it reads supplier quotations (PDF, Excel, scans, email), compares them fairly, predicts which supplier will deliver late, drafts negotiation emails, and raises the purchase order with GST, stopping for a human's approval before anything is sent or ordered.

56 API endpoints **204** automated tests passing **10** agents **3** AI vendors (online)

10,324 real shipments (USAID) for ML

---

01 How it works

A buyer creates a procurement request. Everything after that can be done by the agents, but a person approves every email and every order.

1
Collect
Quotations arrive by upload or Gmail.

2
Extract
PDF text, or OCR for scans; the AI fills a structured form and fixes its own mistakes once.

3
Match
Item names are normalised and fuzzy-matched to the request.

4
Compare
Landed cost, delivery, payment terms and reliability are scored and weighted.

5
Assess risk
The ML model predicts each supplier's chance of late delivery.

6
Negotiate
An agent drafts an email; a critic and a guardrail check it.

7
Approve
The run pauses and waits for a person.

8
Order
A PO with CGST/SGST or IGST is created as a PDF and tracked until delivery.

02 Tech stack

Versions are the exact pins in `backend/requirements.txt` and `frontend/package.json`.

LAYER	TECHNOLOGY	FULL FORM / WHAT IT IS	VERSION	USED FOR
BACKEND				
Language	Python	General-purpose programming language	3.12.9	All backend and ML code

LAYER	TECHNOLOGY	FULL FORM / WHAT IT IS	VERSION	USED FOR
Web framework	FastAPI	Python framework for building APIs (Application Programming Interfaces), with automatic validation and docs	0.115.6	The 56 REST endpoints
Server	Uvicorn	ASGI (Asynchronous Server Gateway Interface) web server	0.34.0	Runs the FastAPI app
Validation	Pydantic / pydantic-settings	Data validation using Python type hints	2.10.4 / 2.7.0	Request/response models; every setting read from `.env`
Database	MongoDB	NoSQL (“Not only SQL”) document database; stores BSON (Binary JSON)	7.0.14	Organisations, quotations, suppliers, POs, agent runs
DB driver	Motor + PyMongo	Asynchronous MongoDB driver for Python	3.6.0 / 4.9.2	Non-blocking database access
Agents	LangGraph	Library for building agent workflows as state graphs	1.2.11	Supervisor, handoffs, pause/resume
HTTP client	HTTPX	Async HTTP (HyperText Transfer Protocol) client	0.28.1	Calls to every AI vendor, Gmail and Drive
Auth	PyJWT, passlib + bcrypt	JWT (JSON Web Token) signing; bcrypt password hashing	2.10.1 / 1.7.4 / 4.2.1	Login tokens, password storage
Encryption	cryptography (Fernet)	Fernet = AES-128-CBC encryption plus HMAC-SHA256 authentication	50.0.1	OAuth refresh tokens encrypted at rest
PDF	PyMuPDF	Python bindings for MuPDF, a PDF (Portable Document Format) engine	1.25.1	Text from PDFs; page images for OCR; PO PDFs
OCR	Tesseract (pytesseract) / RapidOCR	OCR (Optical Character Recognition) engines; RapidOCR runs when Tesseract isn't installed	0.3.13 / 1.4.4	Reading scanned quotations
Spreadsheets	pandas, openpyxl	Tabular data analysis; Excel (XLSX) reader	2.2.3 / 3.1.5	Excel quotations, datasets
Images	Pillow	Python Imaging Library fork	11.0.0	Image pre-processing for OCR
Fuzzy matching	RapidFuzz	Fast string similarity (Levenshtein-based)	3.11.0	Matching quoted items to requested items
MACHINE LEARNING				
ML library	scikit-learn	Python machine-learning library	1.9.1	Random Forest, Isolation Forest, TF-IDF, calibration
Numerics	NumPy, SciPy	Numerical Python; Scientific Python	2.5.2	Arrays, sparse matrices
Model storage	joblib	Saves and loads trained Python models	1.6.0	`reliability_model.joblib`
FRONTEND				
UI library	React	JavaScript library for component-based UIs (User Interfaces)	19.2	All pages
Language	TypeScript	JavaScript with static types	6.0	Type-safe frontend code
Build tool	Vite	“Fast” (French): dev server and bundler	8.2	Dev server, production build
Styling	Tailwind CSS	Utility-first CSS (Cascading Style Sheets) framework	3.4	All styling
Components	Radix UI + CVA	Accessible unstyled components; CVA = Class Variance Authority	1.x–2.x	Dialogs, menus, tabs, tooltips
Data fetching	TanStack React Query	Server-state caching library	5.102	API calls, caching, refetching
Routing	React Router	Client-side page routing	7.18	SPA (Single-Page Application) navigation
Charts	Recharts	React charting library	3.10	Spend and forecast charts

LAYER	TECHNOLOGY	FULL FORM / WHAT IT IS	VERSION	USED FOR
Icons / lint	lucide-react, oxlint	Icon set; Rust-based JavaScript linter	1.34 / 1.79	Icons; code checks
AI AND INTEGRATIONS				
LLMs	Gemini, Grok, Qwen	LLM (Large Language Model) vendors: Google, xAI, Alibaba (Ollama supported for local use, not used)	API	Extraction, agents, council
Email	Gmail API + OAuth 2.0	OAuth = Open Authorization	v1	Collecting quotation emails
Storage	Google Drive API	Scope `drive.file` : only files the app creates	v3	Saving PO PDFs
TESTING AND TOOLING				
Tests	pytest, pytest-asyncio	Python test framework, with async support	8.3.4 / 0.25.0	204 backend tests
Version control	Git + GitHub	Distributed version control; hosting	—	Repository on `main`
Containers	Docker Compose	Multi-container definition	—	Optional one-command setup

03 AI agents: exactly what each one does

Here an “agent” means a piece of the program with **one job, its own tools** (functions it can call that read or change the database), and **a rule for when it runs**. Some use an LLM to decide or write. Others are deliberately plain code, because arithmetic and tax should never be guessed.

AGENT	RUNS WHEN	EXACTLY WHAT IT DOES	READS / CALLS	WRITES	LLM?
Supervisor	A run starts (button or Monitor)	Each turn it (1) works out which actions policy allows now, e.g. no purchase order before comparison and risk check; (2) asks the LLM to pick one and give a one-line reason; (3) if the pick isn't allowed, or the reply is broken, uses the policy's first choice; (4) hands off to that agent. It stops after 12 steps or when it must wait for approval.	Run state (what's done, what's known)	Every decision and reason in `agent_runs`	Yes, bounded. Policy fallback
Extraction	A quotation file arrives	Gets text from the PDF (OCR if it's a scan), has the LLM fill a fixed schema (supplier, items, qty, unit price, GST %, delivery date, payment terms), then validates it: totals add up, GST rate is legal, dates are real. If there are errors it sends them back to the LLM **once** to re-read, and keeps whichever version has fewer errors.	File, validator	Structured quotation + list of problems	Yes. Rule-based parser fallback
Comparison	Supervisor hands off	Loads every quotation for the request, computes landed cost (price + GST + transport), scales each criterion 0–1, applies the weights and ranks the suppliers.	Quotations, suppliers	Comparison table	No (pure maths)
Recommendation	After comparison	Writes a plain-English reason why the top supplier is recommended and what the trade-offs are. It can be handed to the LLM council.	Comparison table	Explanation on the comparison	Yes / council. Template fallback
Risk	Before money is committed	Loads the organisation's delivered orders, builds the 10 features for the top supplier and runs **our own trained Random Forest**. Medium or high risk raises a “concern”, which sends the run to negotiation first.	Delivered POs, ML model	Late probability, risk level, reasons	No (ML model)

AGENT	RUNS WHEN	EXACTLY WHAT IT DOES	READS / CALLS	WRITES	LLM?
Negotiation	Supervisor hands off	A tool-using loop, max 4 rounds. The LLM *chooses* which evidence to fetch: `supplier_performance` (late orders, predicted risk), `price_history` (what we paid before), `past_negotiations`. Then it writes the email and calls `submit_draft`. The **guardrail** rejects any mention of other suppliers or their prices, the LLM rewrites, and the **Critic** reviews the result.	4 tools, the org's own records	Email draft, status "awaiting approval", evidence used	Yes. Template fallback
Critic	A draft is ready	Reads the draft as a reviewer would and rejects it for invented figures, other suppliers named, impossible promises, rudeness or an empty message. A rejection replaces the AI draft with the safe template.	The draft	Approved / issues	Yes / council
Purchase Order	Only after a person approves	Builds PO lines from the chosen quotation, numbers the PO, applies CGST+SGST (same state) or IGST (different states), marks the request "awarded" and creates the PDF.	Quotation, supplier, org profile	Purchase order	No (tax is never guessed)
Monitor	By itself: a quotation finishes processing; every 60 min	(1) When a request has 2+ usable quotations, it starts the Supervisor without anyone pressing a button. (2) When a PO passes its delivery date undelivered, it drafts a follow-up email (one per order, never repeated).	Quotations, POs	New runs, follow-up drafts	No (event rules)
Assistant	A user asks in chat	Answers questions by calling 9 tools: list requests, get request, search quotations, get quotation, get comparison, get supplier, spend summary, draft RFQ, draft negotiation. Max 6 tool rounds; only the user's own organisation's data.	9 tools	Answers; drafts (never sent)	Yes (tool calling)

How is this different from an AI wrapper?

An **AI wrapper** is a text box in front of someone else's model: the user types, the app forwards the prompt to ChatGPT or Gemini, and shows whatever comes back. The model is the whole product.

	AI WRAPPER	THIS PROJECT
Who starts it	A person types a prompt	Events: the Monitor agent starts runs when quotations arrive or deliveries go overdue
What the AI produces	Text on a screen	Actions: tool calls that fetch evidence and create drafts and orders in the database
How many models	One call to one model	10 agents with separate roles and handoffs; 3 vendors; a council that cross-checks
Where facts come from	The model's memory, so it can invent numbers	The organisation's own records via retrieval tools; the HSN answer must exist in the official GST master
Checking	None; you trust the reply	Validator + self-correction, deterministic guardrail, Critic agent, policy-bounded routing
Memory	Forgets after the chat	Runs are saved; a run paused for approval resumes days later from the same point
Own ML	None	Random Forest trained on 10,324 real shipments; Isolation Forest; forecasting
Maths and tax	Asked of the LLM	Plain code: scoring, landed cost, GST split, PO totals
Safety	Whatever the model does	Nothing is sent or ordered without a person's approval

The unplug test: remove every API key. A wrapper does nothing. This system still extracts quotations (rule-based parser), compares and ranks suppliers, predicts late delivery (its own ML model), drafts template emails and creates GST purchase orders. The LLM makes it smarter, but it is a component, not the product.

Pause and resume: a run's state is saved in the `agent_runs` collection. Approving it continues from where it stopped.

04 LLM vendors and the council

Every vendor uses the same OpenAI-compatible chat format, so one HTTPX client talks to all of them, with no vendor SDK. Each task can use a different model, written as `vendor:model`.

VENDOR	COMPANY	ENDPOINT	KEY SETTING	PLANNED ROLE
Gemini	Google	`generativelanguage.googleapis.com/v1beta/openai`	`GEMINI_API_KEY`	Default model, extraction, assistant, council monitor
Grok	xAI	`api.x.ai/v1`	`GROK_API_KEY`	Supervisor, negotiation, council member
Qwen (Tongyi Qianwen)	Alibaba Cloud (DashScope / Model Studio)	`dashscope-intl.aliyuncs.com/compatible-mode/v1`	`QWEN_API_KEY`	HSN code choice, council member
Ollama	Runs open models locally	`localhost:11434/v1`	none	Supported, not used: optional local install

How the LLM council decides

1
Answer
Each member (Grok, Qwen, Gemini) answers on its own.

2
Peer review
Each reads the others' answers, labelled only “Member A/B” so it can't favour a famous model, and revises its own.

3
Decide
The monitor model weighs the revised answers and writes the final one.

Safeguards: a member that fails is dropped (at least 2 answers are required). Members never silently switch to a backup model. If the monitor fails twice, the first member's revised answer is used and the result is marked “degraded”. The full transcript is saved.

05 Machine-learning models

1 · Supplier late-delivery risk (headline result)

Dataset: USAID SCMS (United States Agency for International Development, Supply Chain Management System) delivery history, 10,324 real shipments. Label: *late* = delivered after the scheduled date. The split is by time, with no shuffling across years: trained on 2006–2013 and tested on 2014–2015.

0.832
ROC-AUC on 2,545 unseen shipments

0.334
PR-AUC (baseline = late rate 0.140)

0.099
Brier score after calibration (was 0.167)

92.4%
of late shipments caught at the screening threshold

PARAMETER	VALUE	MEANING
Candidates compared	Logistic Regression, Random Forest, Hist Gradient Boosting	Best by cross-validated PR-AUC wins
CV scores (ROC-AUC / PR-AUC)	LR 0.775 / 0.265 · **RF 0.883 / 0.513** · HGB 0.876 / 0.492	Random Forest selected
`n_estimators`	300	Number of trees
`max_depth`	8	Maximum tree depth
`min_samples_leaf`	5	Minimum shipments per leaf
`class_weight`	balanced_subsample	Corrects for late shipments being rare
`random_state`	42	Reproducible results
Cross-validation	5-fold stratified	Every fold keeps the same late rate
Calibration	`CalibratedClassifierCV`, sigmoid (Platt scaling)	Turns scores into real probabilities
Decision threshold	0.291 (F1-optimal) → precision 0.358, recall 0.553	Balanced operating point (shipped)
Screening threshold	0.145 (F2-optimal) → precision 0.317, recall 0.924	Catch almost every late delivery
Features (10)	`has_history, prior_count, prior_on_time_rate, prior_mean_delay_days, recent_late_rate, days_since_previous, log_order_value, log_quantity, shipment_mode, fulfil_via`	
Leakage controls	Features use only deliveries known 30 days before the scheduled date (`knowledge_gap_days=30`); recent window = last 10 shipments; post-delivery fields such as freight and insurance are excluded.	
Retraining	On the organisation's own delivered POs once there are at least 50 (`ML_MIN_RETRAIN_ORDERS`)	

How much training did it get? (There are no epochs)

Epochs belong to neural networks, which pass over the data again and again with gradient descent. These models are not trained that way. The honest equivalents are: **number of trees** for Random Forest, since each tree is built once on a random resample; **boosting rounds** for gradient boosting, the closest thing to epochs because each round corrects the last; and **solver iterations** for logistic regression. All curves below use 5-fold cross-validated PR-AUC on the **training years only (2006–2013)**, so the 2014–2015 test set was never used to tune anything. Script: `python -m ml.training_curves`.

TRAINING STEP	MODEL FITS	WHAT HAPPENS
Model selection	15	3 candidates × 5 folds

TRAINING STEP	MODEL FITS	WHAT HAPPENS
Threshold choice (calibrated, out-of-fold)	25	5 outer folds × 5 calibration folds
Final calibrated model	5	5 Random Forests, each calibrated on its held-out fold; their average is the shipped model
Uncalibrated reference	1	Only to measure what calibration improved
Total	**46**	36 of them Random Forests × 300 trees = **10,800 trees built**; the shipped model holds **1,500 trees** (5 × 300)

RANDOM FOREST: TREES	10	25	50	100	200	300 ✓	500
CV PR-AUC	0.494	0.503	0.506	0.508	0.508	**0.513**	0.515

GRADIENT BOOSTING: ROUNDS	10	50	100	300 ✓	500	800	827 (PEAK)	1000
CV PR-AUC	0.389	0.444	0.451	0.492	0.519	0.531	**0.533**	0.529

RANDOM FOREST: MAX DEPTH	2	4	6	8 ✓	10	12	UNLIMITED
Shuffled CV (years mixed)	0.350	0.399	0.453	0.513	0.552	0.570	**0.580**
Train 2006–11, score 2012–13	**0.213**	0.206	0.185	0.167	0.171	0.161	0.160

✓ = the value used in the shipped model. Logistic regression converged in **31 of 2,000** allowed solver iterations.

What these numbers show, honestly:

- **Trees: on the plateau.** Almost all the gain comes by 50 trees. Going from 300 to 500 adds only 0.002, so 300 is enough.
- **Boosting: stopped too early.** At 300 rounds it was still improving. It peaks at round 827 (0.533), above the Random Forest in this cross-validation.
- **Depth: the peak depends on how you validate.** With shuffled folds, deeper is always better. When the model is trained on earlier years and scored on later ones, which is how it is actually used, **shallow trees win (depth 2)**. Shuffled folds let a deep model memorise a supplier's behaviour in a given period. That is an optimistic bias, and it is the reason the final test was done on later, unseen years (ROC-AUC 0.832).
- **Next step:** tune depth and boosting rounds with time-ordered (forward-chaining) validation instead of shuffled folds. The model was not changed before the review because re-tuning would mean looking at the test set a second time.

2 · Price anomaly detection

Model	Isolation Forest (`n_estimators=200` , `random_state=42`) plus Tukey's fence (1.5 × IQR, Inter-Quartile Range) for prices outside the past range
Minimum history	20 past prices of the same product
Thresholds	“Possible anomaly” below the 10th percentile of past scores; “High anomaly” below the 2nd
Result on SCMS	9,998 prices assessed; 9.3% flagged (7.5% possible, 1.8% high). There are no ground-truth labels, so this is a flag rate, not an accuracy.

3 · Demand forecasting

Methods	Linear trend (Linear Regression) and seasonal-naive (repeat the same month last year)
Selection	Backtested on the last months (up to 6 held out); lowest sMAPE (symmetric Mean Absolute Percentage Error) wins
Parameters	`MIN_MONTHS=12` , `MAX_HOLDOUT=6` , `SEASON=12` , default horizon 3 months

4 · HSN code suggestion (Indian data)

Data	Official GST portal HSN master: 21,935 codes. Benchmark: 114 Indian quotation-style items (29 dev / 85 held-out test)
Stage 1	TF-IDF (Term Frequency–Inverse Document Frequency), character 3–5-grams plus word 1–2-grams, cosine similarity; Indian trade shorthand expanded (MS, GI, TMT, ERW, OPC, MCB, sqmm…)
Stage 2	An LLM picks from the top 30 candidates; its answer must exist in the official master
Held-out results	Text matching alone: **54.1%** top-1, 76.5% correct chapter. Correct answer is in the shortlist of 30 for **97.6%** of items. LLM stage: measured once API keys are in

06 Scoring and matching

Each criterion is scaled 0–1 across the quotations being compared: lower is better for landed cost and delivery days, higher is better for payment credit days. The weights come from settings and are normalised to sum to 1.

CRITERION	WEIGHT	SCALING
Price (landed cost = price + GST + transport)	0.40	1 − (x − min)/(max − min)
Delivery time	0.20	1 − (x − min)/(max − min)
Payment terms (credit days; advance = 0)	0.15	(x − min)/(max − min)
Supplier reliability (ML)	0.15	On-time probability
Other (completeness, warranty…)	0.10	Deductions for missing fields

Item matching score = 0.6 × max(token-sort, token-set ratio) + 0.2 × partial ratio + 0.2 × plain ratio. At **≥ 0.86** the match is automatic, at **≥ 0.62** it is matched but flagged for review, and below that it goes to manual review.

07 All configuration parameters

Every value is read from `backend/.env` , nothing is hard-coded, and these are the defaults. Keys are left blank here; the real `.env` is git-ignored and never pushed.

SETTING	DEFAULT	WHAT IT CONTROLS
APP AND DATABASE		
`APP_NAME`	Autonomous Procurement Assistant	Name shown in the app and emails
`API_V1_PREFIX`	/api/v1	URL prefix of every endpoint
`ENVIRONMENT / DEBUG`	development / true	Outside development, a weak JWT secret is refused
`MONGODB_URI`	mongodb://127.0.0.1:27017	Database address
`MONGODB_DB_NAME`	procurement_assistant	Database name
SECURITY		

SETTING	DEFAULT	WHAT IT CONTROLS
JWT_SECRET_KEY	(must be set, ≥ 32 chars)	Signs login tokens
JWT_ALGORITHM	HS256	HMAC with SHA-256
ACCESS_TOKEN_EXPIRE_MINUTES	10080	Login lasts 7 days
TOKEN_ENCRYPTION_KEY	(derived if empty)	Fernet key for OAuth tokens
CORS_ORIGINS	localhost:5173	Which websites may call the API (CORS = Cross-Origin Resource Sharing)
STORAGE AND DOCUMENTS		
STORAGE_BACKEND	local	Where uploaded files are kept
STORAGE_LOCAL_PATH	../storage	Folder for uploads
MAX_UPLOAD_SIZE_MB	25	Largest accepted file
OCR_ENGINE	auto	auto / tesseract / rapidocr / none
TESSERACT_CMD	(blank)	Path to tesseract.exe if it isn't on PATH
PDF_TEXT_MIN_CHARS_PER_PAGE	60	Fewer characters than this means a scanned page, so OCR is used
PROCESSING_WORKER_CONCURRENCY	2	Documents processed in parallel
AI VENDORS		
AI_PROVIDER	gemini	Default vendor: gemini / grok / qwen / ollama / openrouter / none
GEMINI_API_KEY · _BASE_URL · _MODEL	— · Google OpenAI-compatible URL · gemini-2.5-flash	Google Gemini
GROK_API_KEY · _BASE_URL · _MODEL	— · https://api.x.ai/v1 · grok-4-fast	xAI Grok
QWEN_API_KEY · _BASE_URL · _MODEL	— · DashScope international · qwen-plus	Alibaba Qwen
OLLAMA_BASE_URL · OLLAMA_MODEL	localhost:11434/v1 · llama3.1	Optional local models, no key (not used)
OPENROUTER_*	(optional)	Optional aggregator, not used by default
LLM_TASK_MODELS	(blank)	Model per task, e.g. `negotiation=grok:grok-4-fast,critic=council`
COUNCIL_MODELS	(blank)	Council members (2+, different vendors)
COUNCIL_MONITOR_MODEL	(blank)	Model that makes the final decision
AI_REQUEST_TIMEOUT_SECONDS	120	Longest wait for one AI answer
SCORING AND MATCHING		
SCORE_WEIGHT_PRICE	0.40	Weight of landed cost
SCORE_WEIGHT_DELIVERY	0.20	Weight of delivery time
SCORE_WEIGHT_PAYMENT	0.15	Weight of payment terms

SETTING	DEFAULT	WHAT IT CONTROLS
SCORE_WEIGHT_RELIABILITY	0.15	Weight of supplier reliability
SCORE_WEIGHT_OTHER	0.10	Weight of everything else
MATCH_AUTO_THRESHOLD	0.86	At or above: matched automatically
MATCH_REVIEW_THRESHOLD	0.62	At or above: matched but flagged; below: manual review
AGENTS AND AUTOMATION		
AGENT_MAX_STEPS	12	Supervisor decisions per run
AGENT_AUTOPILOT	true	Monitor starts runs by itself (drafts still need approval)
AGENT_AUTOPILOT_MIN_QUOTATIONS	2	Quotations needed before autopilot starts
AGENT_WATCHDOG_INTERVAL_MINUTES	60	How often overdue deliveries are checked
ASSISTANT_MAX_TOOL_ROUNDS	6	Tool calls per chat answer
EXTRACTION_SELF_CORRECTION	true	Re-read a document once when validation finds errors
NEGOTIATION_DEFAULT_DISCOUNT_PCT	5.0	Target discount when none is given
NEGOTIATION_WEAK_CRITERION_SCORE	0.5	Delivery/payment scores below this get negotiated
ML_MIN_RETRAIN_ORDERS	50	Delivered POs needed to retrain the risk model
GMAIL AND GOOGLE DRIVE		
GOOGLE_CLIENT_ID / _SECRET	(blank)	OAuth client from Google Cloud Console
GOOGLE_REDIRECT_URI	…/api/v1/channels/gmail/callback	Where Google returns after login
GMAIL_SCOPES	gmail.readonly	Read-only mailbox access
GMAIL_SYNC_QUERY	last 30 days, subject quotation/quote/RFQ/…	Which emails are imported
GMAIL_MAX_MESSAGES_PER_SYNC	25	Emails per sync
GMAIL_SYNC_INTERVAL_MINUTES	15	Automatic collection interval (0 = off)
GOOGLE_DRIVE_ENABLED	true	“Save to Drive” on POs
GOOGLE_DRIVE_FOLDER_ID	(blank)	Optional Drive folder for POs
FRONTEND_URL	http://localhost:5173	Where OAuth sends the user back

08 Full forms

SHORT	FULL FORM	WHERE IT APPEARS
AI / ML	Artificial Intelligence / Machine Learning	Throughout
LLM	Large Language Model	Gemini, Grok, Qwen, Llama
API	Application Programming Interface	Backend endpoints, vendor APIs
REST	Representational State Transfer	API style
HTTP / HTTPS	HyperText Transfer Protocol (Secure)	All network calls
JSON	JavaScript Object Notation	API data, AI replies
BSON	Binary JSON	MongoDB's storage format
NoSQL / SQL	Not only SQL / Structured Query Language	MongoDB is a NoSQL database
URI / URL	Uniform Resource Identifier / Locator	Database and API addresses
ASGI	Asynchronous Server Gateway Interface	Uvicorn
JWT	JSON Web Token	Login
HS256 / HMAC	HMAC-SHA-256 / Hash-based Message Authentication Code	JWT signing
SHA	Secure Hash Algorithm	Signing, dataset fingerprint
AES / CBC	Advanced Encryption Standard / Cipher Block Chaining	Fernet token encryption
OAuth	Open Authorization	Gmail and Drive login
CORS	Cross-Origin Resource Sharing	Browser security
SDK	Software Development Kit	Not needed: one HTTP client for all vendors
UI / UX	User Interface / User Experience	Frontend
SPA	Single-Page Application	React frontend
HTML / CSS	HyperText Markup Language / Cascading Style Sheets	Frontend
TS / TSX	TypeScript / TypeScript + JSX (JavaScript XML)	Frontend source
CVA	Class Variance Authority	Component styles
PDF	Portable Document Format	Quotations, POs
OCR	Optical Character Recognition	Scanned quotations
XLSX / CSV	Excel Open XML Spreadsheet / Comma-Separated Values	Quotations, datasets
EML / MIME	Email message file / Multipurpose Internet Mail Extensions	Downloadable draft emails
PO	Purchase Order	Final order document
RFQ	Request for Quotation	Emails asking suppliers for prices
GST	Goods and Services Tax	Indian tax on POs
CGST / SGST / IGST	Central / State / Integrated GST	Intra-state vs inter-state tax
GSTIN	GST Identification Number	Organisation profile
HSN	Harmonized System of Nomenclature	Goods codes on GST invoices
SAC	Services Accounting Code	Service codes in the GST master

SHORT	FULL FORM	WHERE IT APPEARS
MSME / SME	Micro, Small and Medium Enterprises	Target users
USAID	United States Agency for International Development	Dataset source
SCMS	Supply Chain Management System	The 10,324-shipment dataset
RDC	Regional Distribution Centre	SCMS internal shipments
RF / LR / HGB	Random Forest / Logistic Regression / Histogram Gradient Boosting	Candidate models
CV	Cross-Validation	Model selection
ROC-AUC	Receiver Operating Characteristic – Area Under the Curve	Ranking quality (0.5 = random, 1 = perfect)
PR-AUC	Precision–Recall Area Under the Curve	Quality on the rare “late” class
F1 / F2	F-score with β = 1 (balanced) / β = 2 (recall weighted more)	Choosing thresholds
TF-IDF	Term Frequency – Inverse Document Frequency	HSN text matching
IQR	Inter-Quartile Range	Tukey's fence for price outliers
sMAPE	symmetric Mean Absolute Percentage Error	Forecast backtest
E2E / TDD	End-to-End (test) / Test-Driven Development	How the code is verified
xAI	Elon Musk's AI company (maker of Grok)	Grok vendor
Qwen	Tongyi Qianwen (Alibaba)	Qwen vendor

09 Status

AREA	STATE	EVIDENCE
All 15 synopsis features	Built	204 tests pass; frontend checked in the browser
Multi-agent system (supervisor + specialists + monitor)	Built	A live run routed by an LLM was recorded
ML late-delivery model	Measured	ROC-AUC 0.832 on 2,545 unseen shipments
Multi-vendor AI (Gemini, Grok, Qwen)	Built	`tests/test_vendors.py`
LLM council with three vendors	Awaiting keys	Code tested; live run pending the Grok and Qwen keys
HSN accuracy with the LLM	Awaiting keys	Text matching 54.1%; shortlist recall 97.6%
Gmail / Drive live test	Awaiting OAuth client	Code tested; shows “not configured” until the client is set

10 Likely review questions

Why is it “multi-agent” and not just one chatbot?

Ten agents each have one job and their own tools. A supervisor hands work between them, and a critic checks another agent's output. The monitor agent acts on its own when quotations arrive or deliveries go overdue.

What if the AI makes a mistake or is down?

Every AI step has a deterministic fallback (policy routing, heuristic extraction, template emails). The guardrail and the critic check drafts, and nothing is sent or ordered without human approval.

Why is accuracy not reported for the risk model?

Only 14% of shipments are late, so a model that always says “on time” would score 86% accuracy and be useless. ROC-AUC, PR-AUC, precision and recall show whether it actually finds late deliveries.

How do you prevent data leakage?

The split is by time (train 2006–2013, test 2014–2015). Features only use deliveries known 30 days before each shipment, and the test set was used once.

Why an LLM council?

Models from different companies make different mistakes. Anonymous peer review and a separate monitor reduce single-model errors, and a failed member is dropped instead of breaking the answer.

Why these datasets?

USAID SCMS is one of the few public datasets with both scheduled and actual delivery dates. The GST HSN master is the official Indian source.

11 Slide check: what the slides say vs what is built

The “Technical Approach” slide and the block diagram match the project except for these five points. Fix them before presenting, because a reviewer who opens the code will notice.

SLIDE SAYS	WHAT THE PROJECT ACTUALLY USES	WRITE THIS INSTEAD
LlamaParse	Not used. **PyMuPDF** reads digital PDFs; OCR reads scans	Tesseract / RapidOCR, PyMuPDF
Tesseract OCR	Supported, but **not installed on the demo laptop**; **RapidOCR** does the OCR (`OCR_ENGINE=auto`)	OCR: Tesseract / RapidOCR
PostgreSQL / MongoDB	**MongoDB only**	MongoDB
XGBoost	Not used. **Random Forest** (risk) and **Isolation Forest** (price anomalies), both from scikit-learn	Random Forest, Isolation Forest (scikit-learn)
Gemini API / Ollama	**Gemini, Grok and Qwen** as an LLM council; Ollama supported but not used	Gemini, Grok, Qwen (LLM council)
Diagram: “Send Emails”	Drafts are approved, then exported as `.eml` or copied and marked sent. Gmail access is **read-only**, so the app never sends mail itself	Approve → export email / save PO
WhatsApp API	Planned, not built (shown as “Planned” in the app)	Keep, marked as future work
Feedback loop	Exists: the model retrains on the organisation's own deliveries (≥ 50), user corrections are stored as a separate layer, and extraction self-corrects	Keep

Worth adding to the stack slide: Vite, LangGraph, RapidFuzz, JWT + bcrypt, pytest (204 tests).

If asked “why not XGBoost / PostgreSQL?”

XGBoost

“We compared three models, including scikit-learn's gradient boosting, which is the same family of algorithm as XGBoost. Random Forest won in cross-validation, so XGBoost would only have added a dependency for the same kind of model.”

PostgreSQL

“Quotations have no fixed shape: every supplier sends different fields. A document database stores them as they come, without a rigid table schema.”

12 Explain it simply

ROC-AUC 0.832: what to tell the judges

Full form: Receiver Operating Characteristic, Area Under the Curve. The name comes from WWII radar operators.

> "If you pick one shipment that arrived late and one that arrived on time, our model gives the late one a higher risk score **83% of the time**. A coin flip would get 50%; a perfect model 100%."

How it is measured: the model gives every shipment a risk between 0 and 1. The ROC curve plots "late shipments caught" against "false alarms" at every possible cut-off, and AUC is the area under that curve, so it rates the model at all thresholds at once.

> "We trained on 7,779 real USAID shipments from 2006–2013 and tested once on 2,545 shipments from 2014–2015 that the model had never seen: ROC-AUC 0.832. We don't report accuracy, because only 14% of shipments are late; a model that always says 'on time' would score 86% accuracy and catch nothing."

IF THEY ASK	ANSWER
Is 0.83 good?	Above 0.8 is generally considered good, and this is on future years, which is harder than a random split.
How many late ones does it catch?	92% at the screening setting.
What's the catch?	Precision is about 32–36%, so roughly 2 in 3 warnings are false alarms. That's acceptable because a warning only means "check with the supplier before ordering".
Anything besides ROC-AUC?	PR-AUC 0.334 against a random baseline of 0.14, so it finds late shipments about 2.4× better than chance.

How OCR works

In our pipeline: PyMuPDF first tries the PDF's own text layer. If a page yields fewer than 60 characters, it is a scan, so the page is turned into an image and sent to OCR.

Tesseract (open source, originally HP, later maintained by Google)

1. **Clean-up:** converts to black-and-white (Otsu thresholding) and straightens the page.
2. **Layout analysis:** finds text blocks, then lines, then words.
3. **Recognition:** since version 4, an LSTM (Long Short-Term Memory) neural network reads each line image left to right and outputs characters.
4. **Language model:** fixes likely mistakes with a dictionary and gives each word a confidence score.

RapidOCR (what runs on the demo laptop)

1. **DBNet** (Differentiable Binarization) finds where the text boxes are.
2. An angle classifier turns upside-down text the right way up.
3. **CRNN** (Convolutional Recurrent Neural Network) reads each box.
4. Models come from Baidu's PaddleOCR and run with ONNX Runtime (Open Neural Network Exchange), so no system install is needed.

> "OCR is pluggable: Tesseract if it's installed, otherwise RapidOCR. The text then goes to the LLM, and a rule-based parser is the backup."

Random Forest: supplier late-delivery risk

- **One decision tree** is a flowchart of yes/no questions, e.g. “Was this supplier late more than 20% of the time? Is it a large order? Air or truck?” One tree tends to memorise its training data.
- **A forest is 300 trees.** Each trains on a **random resample** of the shipments and looks at a **random subset of features** at each question, so each tree sees the problem differently. They vote, and the share voting “late” becomes the risk probability.
- **Analogy:** asking 300 experienced buyers who each saw different past orders, then averaging their opinions.
- **Settings:** 300 trees, max depth 8, at least 5 shipments per leaf, class weighting (late is rare), 5-fold cross-validation, calibrated so that “30% risk” really means about 30%.
- **Why this model:** it handles mixed number and category features and non-linear patterns, and it beat Logistic Regression and Gradient Boosting in cross-validation.

Isolation Forest: price anomalies

- **The idea:** unusual things are few and different, so they are easy to isolate.
- **How:** it builds random trees that split the prices at random points. A normal price sits in a crowd and needs many splits before it is alone; a strange price (₹900 when everyone quotes ₹280) is isolated in 2–3 splits. **Fewer splits means more anomalous.**
- **No labels needed** (unsupervised), which matters because nobody has marked which past prices were wrong.
- **Settings:** 200 trees per product, only with 20+ past prices; below the 10th percentile score is “possible anomaly”, below the 2nd “high”. Tukey's fence (1.5 × IQR) also catches prices outside the past range, which Isolation Forest handles poorly.
- **Result on USAID data:** 9.3% of 9,998 prices flagged. This is a flag rate, not an accuracy, because there is no ground truth.